

Git Branching Strategies Explained Simply For Beginners

Understanding how software teams structure Git branches is essential for writing safe code at work. This practical guide covers the standard Git workflows every beginner engineer uses on real projects.

What you'll learn:

1. Working Safely With Feature Branches
2. Handling Emergency Production Hotfixes
3. Updating Your Branch With Team Changes
4. Cleaning Up Workspace After Merging

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

1. Working Safely With Feature Branches

Never write code directly on the main branch in a company setting. Always start by fetching the latest main branch from your remote repository. Then, create a named feature branch using clear prefixes like 'feature/' or 'fix/'. This isolates your changes safely so you can experiment without breaking live application code.

```
$git checkout main  
git pull origin main  
git checkout -b feature/add-user-login
```

Cloud & DevOps Daily

Git Branching Strategy: What to Use at Work

Monday, August 17, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

2. Handling Emergency Production Hotfixes

When a severe bug crashes production, you cannot wait for normal feature release schedules. Hotfix branches are branch cutouts created directly from the live main branch. You fix the bug, test it thoroughly, and merge it immediately back into main. Afterward, sync this fix into active development branches so the bug does not return.

```
$git checkout main
git checkout -b hotfix/fix-auth-crash
# make bug fix
git commit -am "fix: resolve auth null pointer crash"
git push origin hotfix/fix-auth-crash
```

Cloud & DevOps Daily

Git Branching Strategy: What to Use at Work

Monday, August 17, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

3. Updating Your Branch With Team Changes

While you build a feature, teammates continue merging their own work into the main branch. If you wait too long to update your branch, you will face massive merge conflicts. Run a git pull rebase regularly on your feature branch. This places your team's newest commits first and applies your custom commits neatly on top.

```
$git checkout feature/add-user-login  
git pull --rebase origin main
```

Cloud & DevOps Daily

Git Branching Strategy: What to Use at Work

Monday, August 17, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

4. Cleaning Up Workspace After Merging

After your pull request gets reviewed, approved, and merged into main on GitHub or GitLab, clean up your environment. Keeping old, merged branches on your machine causes confusion and clutter. Delete the feature branch locally and remove the tracking branch from your remote server.

```
$git checkout main
git pull origin main
git branch -d feature/add-user-login
git push origin --delete feature/add-user-login
```


Cloud & DevOps Daily

Git Branching Strategy: What to Use at Work

Monday, August 17, 2026

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- + Use clear naming conventions like feature/ticket-id or fix/description.
- + Sync your local feature branch with main daily to avoid merge conflicts.
- + Keep pull requests small so team members can review them quickly.
- + Delete feature branches both locally and remotely right after merging.
- + Never run force push (git push -f) directly against the main branch.

Watch Out For

- ! Pushing code directly to main instead of using feature branches and PRs.
- ! Leaving feature branches active for weeks without syncing with main.
- ! Committing API keys or secrets directly into Git branch history.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh